

Schema Register

Visual reference for the Alumni Relations tables defined in

`alumniDatabaseSchema.md` — PostgreSQL DDL, row-level multi-tenant via

`school_id`. `schools` and `users` are stubs owned by other modules, shown here only so foreign keys resolve.

10	2	5	5
ALUMNI TABLES	STUB DEPENDENCIES	FEATURE DOMAINS	V2 TABLES, UNSPECCED

Relations Directory Class groups Events Mentorship Donations

V2 roadmap

`pk` primary key `fk` foreign key → target `uniq` unique constraint `check` value constraint bold mono = not null

§1 How the tables relate

Every alumni-facing table traces back to a school. Child tables carry a denormalized `school_id` for direct tenant-scoped queries even where it's derivable through a join — see the DDL doc §2.

PARENT		CHILD	VIA COLUMN	CARDINALITY
<code>schools</code>	→	<code>alumni_profiles</code>	<code>school_id</code>	1 – many
<code>users</code>	→	<code>alumni_profiles</code>	<code>user_id</code>	1 – 1
<code>alumni_profiles</code>	→	<code>alumni_verification_requests</code>	<code>alumni_profile_id</code>	1 – many (1 pending)
<code>schools</code>	→	<code>class_groups</code>	<code>school_id</code>	1 – many
<code>class_groups</code>	→	<code>class_group_posts</code>	<code>class_group_id</code>	1 – many
<code>class_group_posts</code>	→	<code>class_group_post_reactions</code>	<code>post_id</code>	1 – many
<code>users</code>	→	<code>class_group_posts</code>	<code>author_id</code>	1 – many
<code>users</code>	→	<code>class_group_post_reactions</code>	<code>user_id</code>	1 – many

PARENT		CHILD	VIA COLUMN	CARDINALITY
schools	→	events	school_id	1 – many
events	→	event_rsvps	event_id	1 – many
users	→	event_rsvps	user_id	1 – many
alumni_profiles	→	mentorship_connections	mentor_id	1 – many
users	→	mentorship_connections	requester_id	1 – many
schools	→	donation_campaigns	school_id	1 – many
donation_campaigns	→	donations	campaign_id	1 – many
users	→	donations	donor_id	1 – many

§2 Alumni Directory

The core entity — one profile per alumnus, plus the admin verification queue behind it. Stub dependencies shown alongside for FK context.

schools			STUB · OTHER MODULE
id	uuid	pk	
name	text	not null	
slug	text	uniq	not null
country	varchar(2)	ISO alpha-2: GM · ZW	
created_at	timestamptz	default now()	

users			STUB · OTHER MODULE
id	uuid	pk	
phone	text	uniq	E.164
name	text	nullable	
role	varchar(20)	alumni · school_admin · super_admin	
avatar_url	text	nullable	
created_at	timestamptz	default now()	

alumni_profiles

CORE ENTITY

id	uuid	pk
user_id	uuid	fk
school_id	uuid	fk
graduation_year	smallint	ch
class_section	varchar(100)	e.g
current_city	varchar(120)	nul
current_country	varchar(2)	ISC
profession	varchar(150)	nul
employer	varchar(150)	nul
bio	text	nul
linkedin_url	text	nul
verification_status	varchar(20)	ch unv
verification_answer	text	hor
is_mentor	boolean	def
mentor_areas	text[]	def
visibility	jsonb	per
deleted_at	timestamptz	sof
created_at / updated_at	timestamptz	def

indexes (school_id, graduation_year) ·
(school_id, current_city) · (school_id) where
is_mentor

alumni_verification_requests

REVIEW LOG

id	uuid	pk
alumni_profile_id	uuid	fk
school_id	uuid	fk der
verification_answer_snap	text	not
status	varchar(20)	ch per
reviewed_by	uuid	fk
reviewer_notes	text	nul
created_at / reviewed_at	timestamptz	rev

uniq (alumni_profile_id) where status='pending'
· **index** (school_id, status)

§3 Class Year Groups

One group per (school, graduation year). Membership is implicit — no membership table — a profile belongs if its year matches.

class_groups		
id	uuid	pk
school_id	uuid	fk → schools.
graduation_year	smallint	not null
name	varchar(50)	auto: "Class of 2015"
created_at	timestamptz	default now()
uniq (school_id, graduation_year)		

class_group_posts		
id	uuid	pk
class_group_id	uuid	fk → class_groups.
school_id	uuid	fk → schools. depends on class_group_id
author_id	uuid	fk → users.id
content	text	checked for profanity
image_url	text	null
deleted_at	timestamptz	soft delete
created_at / updated_at	timestamptz	default now()
index (class_group_id, created_at desc) where not deleted		

class_group_post_reactions		
id	uuid	pk
post_id	uuid	fk → class_group_posts.id
user_id	uuid	fk → users.id
type	varchar(20)	default 'like'
created_at	timestamptz	default now()
uniq (post_id, user_id) — no school_id, always reached via post_id		

§4 Events

Reunions, career days, fundraisers — with RSVP tracking.

events			event_rsups		
id	uuid	pk	id	uuid	pk
school_id	uuid	fk	event_id	uuid	fk
created_by	uuid	fk	school_id	uuid	fk depend
title	varchar(200)	not	user_id	uuid	fk
description	text	nul	status	varchar(20)	che goi
event_date	timestampz	not	created_at / updated_at	timestampz	defa
end_date	timestampz	ch eve	uniq (event_id, user_id) · index (event_id, status)		
location	varchar(255)	nul			
location_type	varchar(20)	phy hyb			
virtual_link	text	nul			
cover_image_url	text	nul			
is_published	boolean	def			
created_at / updated_at	timestampz	def			
index (school_id, event_date) where published					

§5 Mentorship – V1.1

Opt-in fields live on `alumni_profiles` (`is_mentor`, `mentor_areas`); this table backs the request/accept flow.

mentorship_connections		
id	uuid	pk
school_id	uuid	fk → schools.id denorm.
mentor_id	uuid	fk → alumni_profiles.id
requester_id	uuid	fk → users.id
area	varchar(50)	career · university · tutoring · general
message	text	nullable
status	varchar(20)	check pending·accepted·declined·closed
created_at / updated_at	timestampz	default now()
index (mentor_id, status) · app layer enforces mentor_id.is_mentor = true		

Amounts stored as integers in smallest currency unit. Financial rows use `ON DELETE RESTRICT`, not cascade — money trails don't get deleted by accident.

donation_campaigns		
id	uuid	pk
school_id	uuid	fk
title	varchar(200)	not null
description	text	null
goal_amount	integer	check (GM)
currency	varchar(3)	check (GM)
deadline	timestampz	null
is_active	boolean	default true
created_at / updated_at	timestampz	default

donations		
id	uuid	pk
campaign_id	uuid	fk, restrict
school_id	uuid	fk, restrict
donor_id	uuid	fk
amount	integer	check
currency	varchar(3)	not null
payment_method	varchar(20)	way
payment_ref	varchar(255)	unique
is_anonymous	boolean	default false
status	varchar(20)	per
created_at / updated_at	timestampz	default
index (campaign_id, status)		

§7 V2 roadmap — names only

Product decisions behind these (messaging model, chapter geography, moderation) aren't settled — no DDL yet, just placeholders.

`job_postings`

A9 — alumni post job opportunities

`alumni_chapters /`
`alumni_chapter_memberships`

A10 — location-based alumni groups

`direct_messages`

A11 — in-platform messaging

`broadcast_messages`

A14 — SMS/WhatsApp bulk broadcast

`(analytics: views, not tables)`

A13 — likely read-model over existing tables